URL Shortener + Analytics

Beginner → Production Backend Learning Project

A step-by-step project guide for someone who currently knows basic CRUD with Node.js, Express.js and React, and wants to learn backend engineering properly.

Start with	Node.js + Express.js + React
Database	PostgreSQL
Cache / Rate Limit	Redis
Containers	Docker
Cloud	AWS
Later	CI/CD + Monitoring + Load Testing

Scope: This version intentionally does NOT include agentic AI or RAG. Those can be added later after the backend foundation is solid.

1. What You Are Building

You will build a small URL-shortening platform similar in concept to Bitly. A user submits a long URL. Your backend creates a short code. When another user opens the short URL, the backend finds the original URL and redirects the browser. You will then add authentication, analytics, Redis, rate limiting, background jobs, Docker, AWS, testing, CI/CD and monitoring step by step.

```
Long URL
https://youtube.com/watch?v=abc123
        |
        v
POST /api/v1/urls
        |
        v
Generate short code: aB72x
        |
        v
https://yourapp.com/aB72x
        |
        v
GET /aB72x
        |
        v
Find original URL
        |
        v
HTTP 302 Redirect
```

The most important rule

Do not try to build the final architecture on day one. Each phase should introduce one new backend concept. You should be able to explain why a component exists before adding the next component.

2. Your Current Starting Point

You already know the basics of CRUD using Node.js, Express.js and React. So the project starts with familiar technology rather than forcing you to learn a completely new backend framework first.

• You know how an Express server receives requests.

• You know basic GET, POST, PUT/PATCH and DELETE operations.

• You have basic frontend knowledge in React.

• You do not need to know Redis, Docker, AWS, queues or advanced system design yet.

3. Technology Stack

Layer	Technology	Why
Frontend	React	Simple interface for creating and managing links
Backend	Node.js + Express.js	Build APIs and learn backend fundamentals
Database	PostgreSQL	Learn relational data, SQL and indexes
Cache	Redis	Learn caching and fast temporary data
Rate limiting	Redis	Protect APIs from excessive requests

Layer	Technology	Why
Background jobs	Node.js worker + Redis queue	Move slow work away from request path
Container	Docker	Package the application consistently
Cloud	AWS	Learn real deployment and infrastructure
CI/CD	GitHub Actions	Automate testing and deployment
Monitoring	CloudWatch + later Prometheus/Grafana	Understand production health

4. Final Learning Path

```
Phase 1   Basic Express API
   ↓
Phase 2   PostgreSQL + SQL
   ↓
Phase 3   Authentication + Authorization
   ↓
Phase 4   Redis Caching
   ↓
Phase 5   Rate Limiting
   ↓
Phase 6   Background Jobs + Queue
   ↓
Phase 7   Click Analytics
   ↓
Phase 8   Docker
   ↓
Phase 9   AWS Deployment
   ↓
Phase 10  Testing
   ↓
Phase 11  CI/CD
   ↓
Phase 12  Monitoring + Load Testing
```

Every phase below contains: what to build, what to learn, and what you should be able to explain before moving forward.

5. Phase 1 — Build the Basic Express Backend

Goal: take your existing CRUD knowledge and turn it into a clean backend API. Do not add Redis or Docker yet.

Core endpoints

```
POST   /api/v1/urls
GET    /api/v1/urls
GET    /api/v1/urls/:id
DELETE /api/v1/urls/:id
GET    /:shortCode
```

Example request

```
POST /api/v1/urls
Content-Type: application/json

{
  "longUrl": "https://youtube.com/watch?v=abc123"
}
```

Example response

```
{
  "shortCode": "aB72x",
  "shortUrl": "http://localhost:3000/aB72x"
}
```

What you learn

• HTTP methods and status codes

• Express routing

• Controllers and services

• Request validation

• Middleware

• Centralized error handling

• Environment variables

• Clean project structure

Suggested beginner structure

```
url-shortener/
■■■ src/
■   ■■■ server.js
■   ■■■ app.js
■   ■■■ routes/
■   ■   ■■■ url.routes.js
■   ■   ■■■ auth.routes.js
■   ■■■ controllers/
■   ■■■ services/
■   ■■■ middleware/
■   ■■■ models/
■   ■■■ config/
■   ■■■ utils/
■■■ tests/
■■■ .env
■■■ .gitignore
■■■ package.json
```

Checkpoint

You should be able to explain what happens from the moment a request enters Express until a response is returned. If you cannot explain router → controller → service → database, stay on this phase.

6. Phase 2 — PostgreSQL and SQL

Replace temporary/in-memory data with PostgreSQL. This is where the project starts teaching real database concepts.

Initial schema

```
users
-----
id
email
password_hash
created_at

urls
----
```

```
id
user_id
long_url
short_code
click_count
created_at
expires_at

click_events
------------
id
url_id
ip_address
user_agent
country
created_at
```

Relationships

```
users 1 ■■■■■■■■■■■■ N urls
urls  1 ■■■■■■■■■■■■ N click_events
```

Learn these database concepts

• Primary keys and foreign keys

• Unique constraints — short_code must be unique

• Indexes — especially short_code and user_id

• SELECT, INSERT, UPDATE and DELETE

• JOINs

• GROUP BY and aggregation

• Transactions

• Pagination

• Query performance

Example query

```
SELECT *
FROM urls
WHERE user_id = $1
ORDER BY created_at DESC
LIMIT 20 OFFSET 40;
```

7. Phase 3 — Authentication and Authorization

Now users can create accounts and manage only their own URLs.

Endpoints

```
POST /api/v1/auth/register
POST /api/v1/auth/login
GET  /api/v1/users/me
```

Authentication flow

```
Register:
password
```

```
   ↓
hash password
   ↓
store password_hash

Login:
email + password
   ↓
compare with hash
   ↓
create access token
   ↓
client sends token on protected requests
```

Learn

• Password hashing

• JWT access tokens

• Authentication middleware

• Authorization — user A cannot delete user B's URL

• Protected routes

• Refresh tokens as a later improvement

8. Phase 4 — Redis Caching

Redis is an in-memory data store. Use it first as a cache, not as a replacement for your primary database.

Without cache

```
GET /aB72x
   ↓
Express
   ↓
PostgreSQL
   ↓
original URL
   ↓
redirect
```

With cache

```
GET /aB72x
   ↓
Express
   ↓
Redis
   ■■■ HIT  → redirect
   ■■■ MISS → PostgreSQL
                 ↓
              Redis SET
                 ↓
              redirect
```

Suggested Redis key

```
url:aB72x
    → https://youtube.com/watch?v=abc123
```

Learn

• Cache hit vs cache miss

• TTL (time to live)

• Cache-aside pattern

• Cache invalidation

• Redis GET/SET

• When caching helps and when it does not

Important exercise

Measure the endpoint before and after caching. Do not just add Redis because a tutorial said to. Understand the performance reason.

9. Phase 5 — Rate Limiting

Protect your API from clients making too many requests. Start with a simple fixed-window limiter.

```
Example:
100 requests per IP
within 60 seconds

request
   ↓
rate limiter
   ■■■ under limit → continue
   ■■■ over limit  → HTTP 429
```

Redis concept

```
rate_limit:<client-ip>
count = number of requests
TTL   = seconds until window resets
```

Then learn

• Fixed Window

• Sliding Window

• Token Bucket

• Leaky Bucket

Start with one algorithm. Understand it completely before implementing the others.

10. Phase 6 — Background Jobs and Queues

Click analytics should not make the redirect unnecessarily slow. Move non-critical work to a background worker.

```
User
 ↓
GET /aB72x
 ↓
Backend
 ■■■ enqueue click event
 ■■■ redirect immediately

Queue
```

```
 ↓
Worker
 ↓
PostgreSQL
```

What to learn

• Why background jobs exist

• Producer and consumer

• Retries

• Failed jobs

• Idempotency

• Job status

You can use a Redis-backed Node.js queue library such as BullMQ. The exact library is less important than understanding the queue model.

11. Phase 7 — Analytics

Now make the project useful to users. Track clicks and provide an analytics page in React.

Endpoint

```
GET /api/v1/urls/:id/analytics
```

Possible response

```
{
  "totalClicks": 15240,
  "uniqueVisitors": 10231,
  "devices": {
    "mobile": 9200,
    "desktop": 6040
  },
  "topCountries": [
    { "country": "India", "clicks": 7230 }
  ]
}
```

• SQL aggregation

• GROUP BY

• Indexes

• Time-range filtering

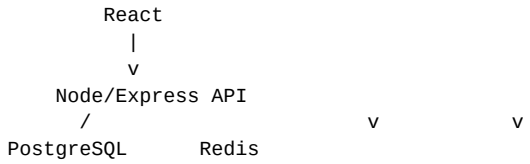
• Pagination

• Simple React charts

12. Phase 8 — Docker

Once the application works locally, package it with Docker so that your application and its dependencies can run consistently.

Local architecture



Files to create

```
Dockerfile
docker-compose.yml
.dockerignore
.env
```

Learn

• Images vs containers

• Dockerfile

• Port mapping

• Volumes

• Networks

• Environment variables

• Docker Compose

• Container health checks

Checkpoint

A new developer should be able to clone the repository, configure the environment file, run Docker Compose and start the application without manually installing PostgreSQL or Redis.

13. Phase 9 — AWS Deployment

Do not begin with a complicated multi-service AWS architecture. First deploy one working containerized backend. Then introduce managed services.

Learning progression

```
Step 1
Local Docker
   ↓
Step 2
Single AWS EC2 instance
   ↓
Step 3
Use AWS RDS for PostgreSQL
   ↓
Step 4
Use ElastiCache for Redis
   ↓
Step 5
Move container deployment toward ECS
   ↓
Step 6
Add ALB + Route 53 + CloudWatch
```

AWS topics

• IAM — permissions and roles

• EC2 — virtual server

• Security Groups — network access rules

• VPC — networking foundation

• RDS — managed PostgreSQL

• ElastiCache — managed Redis

• ECR — Docker image registry

• ECS — container orchestration

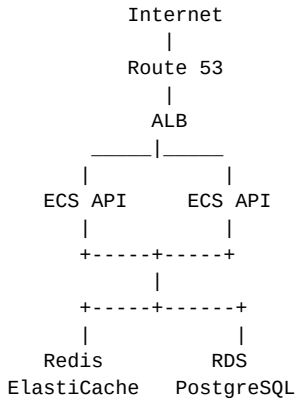
• ALB — load balancing

• Route 53 — DNS

• CloudWatch — logs and metrics

• S3 — object storage for future needs

Production-style architecture



14. Phase 10 — Testing

Testing should grow with the project. You do not need hundreds of tests at the beginning.

Start with

• Unit tests for URL generation and validation

• Unit tests for rate-limiter logic

• API tests for register/login

• API tests for create/get/delete URL

• Integration tests involving PostgreSQL and Redis

Example test cases

```
POST /api/v1/urls
```

```
  ✓ valid URL creates short code
  ✓ invalid URL returns 400
  ✓ duplicate short code is handled

  GET /:shortCode
  ✓ existing code redirects
  ✓ missing code returns 404

  Rate limit
  ✓ request under limit succeeds
  ✓ request over limit returns 429
```

15. Phase 11 — CI/CD

Automate the repetitive parts of development. Every push should be checked before you deploy it.

```
git push
   ↓
GitHub
   ↓
GitHub Actions
   ↓
Install dependencies
   ↓
Run tests
   ↓
Build Docker image
   ↓
Push image to ECR
   ↓
Deploy to AWS
```

• GitHub Actions basics

• Secrets and environment variables

• Automated tests

• Docker image builds

• Deployment workflow

• Rollback thinking

16. Phase 12 — Monitoring and Load Testing

A backend is not finished when it returns the correct response. In production you need to know whether it is healthy.

Track

• Request count

• Response latency

• Error rate

• CPU and memory

• Database connections

• Redis hit rate

• Rate-limit violations

• Queue size and failed jobs

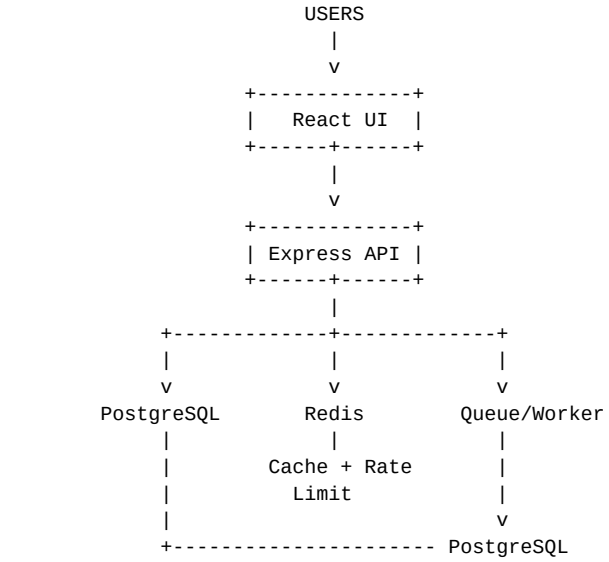
Useful metrics

```
Average latency
P50 latency
P95 latency
P99 latency
HTTP 4xx count
HTTP 5xx count
Redis cache hit ratio
Database query latency
```

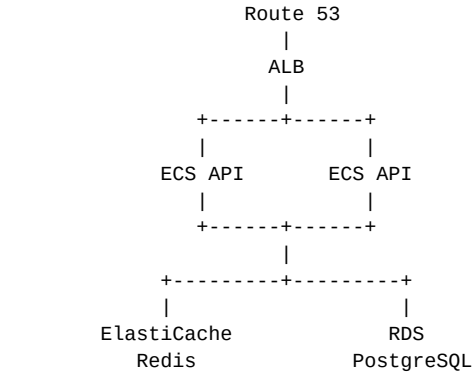
Load testing

Use a tool such as k6 or Locust to send many requests and observe what breaks first. The goal is to connect theory to real measurements.

17. Final Project Architecture



AWS version:



Monitoring:
ECS / RDS / Redis → CloudWatch
Later → Prometheus + Grafana

18. Complete Folder Structure

```
url-shortener/
■■■ client/
■   ■■■ React application
■
■■■ server/
■   ■■■ src/
■   ■   ■■■ app.js
■   ■   ■■■ server.js
■   ■   ■■■ config/
■   ■   ■■■ routes/
■   ■   ■   ■■■ auth.routes.js
■   ■   ■   ■■■ url.routes.js
■   ■   ■   ■■■ analytics.routes.js
■   ■   ■■■ controllers/
■   ■   ■■■ services/
■   ■   ■■■ repositories/
■   ■   ■■■ models/
■   ■   ■■■ middleware/
■   ■   ■   ■■■ auth.js
■   ■   ■   ■■■ rateLimiter.js
■   ■   ■   ■■■ errorHandler.js
■   ■   ■■■ cache/
■   ■   ■   ■■■ redis.js
```

```
■   ■   ■■■ workers/
■   ■   ■   ■■■ analytics.worker.js
■   ■   ■■■ utils/
■   ■■■ tests/
■
■■■ Dockerfile
■■■ docker-compose.yml
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

19. API Map

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Create account
POST	/api/v1/auth/login	Login
GET	/api/v1/users/me	Current user
POST	/api/v1/urls	Create short URL
GET	/api/v1/urls	List user's URLs
GET	/api/v1/urls/:id	Get one URL
DELETE	/api/v1/urls/:id	Delete URL
GET	/api/v1/urls/:id/analytics	View analytics
GET	/:shortCode	Redirect

20. What You Should Know Before Moving to the Next Phase

After	You should be able to explain
Phase 1	How Express routing, middleware, controllers and services work
Phase 2	Why PostgreSQL needs indexes, relationships and constraints
Phase 3	Authentication vs authorization and how protected routes work
Phase 4	Why a cache improves repeated reads and what cache invalidation means
Phase 5	How your chosen rate-limit algorithm works
Phase 6	Why a queue/worker is useful and how retries work
Phase 7	How analytics queries aggregate click data
Phase 8	How Docker packages and connects application services
Phase 9	How your application reaches users on AWS
Phase 10	What unit, API and integration tests protect
Phase 11	How code moves from git push to deployment
Phase 12	How to identify latency, errors and resource problems

21. Beginner Rules for This Project

• Build one feature at a time. Avoid copying the final architecture.

• When you learn a technology, first understand the problem it solves.

• Use PostgreSQL as the source of truth. Redis is not your primary database.

• Measure performance before and after adding caching.

• Start rate limiting with one simple algorithm.

• Keep production credentials out of Git.

• Learn Docker locally before trying to debug Docker on AWS.

• Deploy a simple version to AWS before introducing load balancing and multiple containers.

• Write tests for important behavior before adding more infrastructure.

• Read your logs. Debugging is a core backend skill.

22. Suggested 8-Stage Study Plan

Stage	Focus	Deliverable
1	Express + PostgreSQL	Working URL shortener
2	Auth + clean architecture	Accounts + protected URLs
3	Redis	Cached redirects
4	Rate limiting	Protected APIs
5	Queue + analytics	Fast redirects + analytics
6	Docker	Full local environment
7	AWS + CI/CD	Public deployment
8	Monitoring + load testing	Production-style system

Later extension: Once this backend is comfortable, the project can be extended with an AI agent and RAG layer. Those are intentionally excluded from this document so that you first build strong backend fundamentals.

23. Final Goal

By completing this project in order, you should move from knowing basic Express CRUD to understanding how a real backend is structured, how data is stored and optimized, how Redis improves performance, how APIs are protected, how background work is processed, how containers run the system, how the application is deployed to AWS, and how a production service is tested and monitored.

Remember: the goal is not to finish the project as quickly as possible. The goal is to understand every layer well enough that you could rebuild it without following a tutorial.